



ENGIN 604 INTRODUCTION TO PYTHON FOR FINANCE

FUNCTIONS - SOLUTION

Professor: *Gabriel E. Cabrera*
Teaching Assistant: *Alex Den Braber*



1 def(x)

A function is declared using the keyword `def` and only runs when it is called. Its main purpose is to avoid code redundancy on the programmer's part. For example, to create a function that multiplies any number (int or float) by two and then computes its square root:

```
def my_function(x): # creating a function must start with def,  
                    # then the name of the function and, in parentheses,  
                    # the arguments it will take  
  
    '''  
    Documentation (docstring):  
    Multiplies the numeric argument by two and then computes the square root.  
    '''  
  
    a = x * 2 # multiplies x by two (body)  
    b = a ** 0.5 # computes the square root of x multiplied by two (body)  
  
    return(b) # the function must end with return so that it "returns"  
              # the final result of the function  
  
# the function is checked  
my_function(4) # (x * 2) ** 0.5 = (4 * 2) ** 0.5
```

```
## 2.8284271247461903
```

Functions can also take more than one parameter (argument):

```
def my_function2(x, y): # def, function name, parameters (arguments)  
    '''  
    Documentation (docstring):  
    Adds the two numeric arguments (x + y).  
    '''  
  
    a = x + y # adds x and y (body)  
  
    return(a) # return  
  
# the function is checked  
my_function2(4, 5) # x + y = 4 + 5
```

```
## 9
```

Functions can also return more than one output:

```
def my_function3(x, y): # def, function name, parameters (arguments)
    '''
    Documentation (docstring):
    Multiplies and adds both arguments (x and y). Returns the multiplication
    and the sum separately.
    '''
    a = x * y # multiplication of x and y (body)
    c = x + y # adds x and y (body)

    return(a, c) # return

# the function is checked
mult_x_y, sum_x_y = my_function3(4, 5) # x * y = 4 * 5 ^ x + y = 4 + 5

mult_x_y

## 20
sum_x_y

## 9
```

1.1 Applications

1. Build a function that, for a number n , returns as *output* the sum of the squares $1^2 + 2^2 + 3^2 + 4^3 + \dots + n^2$.

```
def sum_of_squares(n):
    total = 0
    for i in range(1,n+1):
        total += i ** 2
    return(total)

sum_of_squares(5)
```

55

2. Build a function that returns as output whether or not a number is divisible by 4.

```
def divisible(n):
    if (n % 4) == 0:
        return(print('The number '+str(n)+' is divisible by 4.'))
    else:
        return(print('The number '+str(n)+' is not divisible by 4.'))

divisible(16)
```

The number 16 is divisible by 4.

3. Build a function that returns the arithmetic mean of a data set.

```
def arithmetic_mean(n):
    output = sum(n) / len(n)
    return(output)

arithmetic_mean([1,2,3,4,5])
```

3.0

4. Build a function that computes the net present value (NPV) of a cash flow.

$$NPV = \sum_{t=0}^T \frac{F_t}{(1+r)^t}$$

Where F_t is the cash flow in period t , r is the discount rate and T is the total number of periods. Consider the following cash flows:

Table 1: Periods & Cash Flows

Period	Cash Flow (\$)
0	-500000
1	100000
2	150000
3	180000
4	200000
5	300000

If the discount rate is 12%, the net present value is:

$$NPV = -500000 + \frac{100000}{(1+0.12)^1} + \frac{150000}{(1+0.12)^2} + \frac{180000}{(1+0.12)^3} + \frac{200000}{(1+0.12)^4} + \frac{300000}{(1+0.12)^5} = 134316.91$$

```
def net_present_value(cash_flow, rate):
    total = 0
    for i, num in enumerate(cash_flow):
        if i == 0:
            total += num
        else:
            total += num / (1 + rate) ** (i)
    return(total)

cash_flows = [-500000, 100000, 150000, 180000, 200000, 300000]

net_present_value(cash_flows, 0.12)

## 134316.91292132728
```

2 Anonymous Functions (Lambda)

Anonymous or lambda functions consist of writing a function in a single statement. They are used through the *keyword* `lambda`, which “tells” Python “an anonymous function is being declared”. They are useful inside a defined function (`def`), for example:

```
seq = [1, 2, 4]

def apply_to_a_list(list_, f):
    return [f(x) for x in list_]

apply_to_a_list(seq, lambda x: x ** 2)

## [1, 4, 16]
```

The function `apply_to_a_list` takes as arguments a list of numbers (`seq`) and a function (`lambda`) that is applied through a *list comprehension*, returning each number of that list squared. They are called anonymous because they are not defined using `def`.

3 Importing Functions

1. Save the functions created in 1.3 and 1.4 in a script named `helper_functions.py`. Then import them into your workspace and check that they can be used in the same way, that is, without having to program them from scratch every time you want to use them.

To load the functions from the file `helper_functions.py`:

```
# it must be in the same directory, if you use * instead of the name of each function  
# all the available functions are loaded  
from helper_functions import arithmetic_mean, net_present_value
```

```
# function from 1.3  
arithmetic_mean([1,2,3,4,5])
```

```
## 3.0
```

```
# function from 1.4  
cash_flows = [-500000,100000,150000,180000,200000,300000]  
  
net_present_value(cash_flows, 0.12)
```

```
## 134316.91292132728
```